

Exercise 8

Advanced Methods for Regression and Classification

Kyzer Gerez

03.12.2025

```
library(datasets)
data("cars")

# split into 2/3 train, 1/3 test
smp_size <- floor(2/3 * nrow(cars))

# set the seed to make partition reproducible
set.seed(1)
train_ind <- sample(seq_len(nrow(cars)), size = smp_size)

train <- cars[train_ind, ]
test <- cars[-train_ind, ]

# sort train and test sets by speed
train <- train[order(train$speed),]
test <- test[order(test$speed),]
```

1. Construct the basis functions from the training set. Visualize the basis functions.

```
lecturespl <- function(x, nknots=2, M=4) {
  # nknots ... number of knots -> placed at regular quantiles
  # M ... M-1 is the degree of the polynomial
  n <- length(x)
  # X will not get an intercept column
  X <- matrix(NA, nrow=n, ncol=(M-1)+nknots)
  for (i in 1:(M-1)){
    X[,i] <- x^i
  }
  # now the basis functions for the constraints:
  quant <- seq(0,1,1/(nknots+1))[c(2:(nknots+1))]
  qu <- quantile(x,quant)
  for (i in M:(M+nknots-1)) {
    X[,i] <- ifelse(x-qu[i-M+1]<0,0,(x-qu[i-M+1])^(M-1))
  }
  list(X=X, quantiles=quant, xquantiles=qu)
}

# added so that basis functions can be evaluated on new x
make_basis <- function(x, nknots=2, M=4) {
  out <- lecturespl(x, nknots, M)
  X <- out$X
```

```

colnames(X) <- paste0("b", seq_len(ncol(X)))
list(X = X, info = out)
}

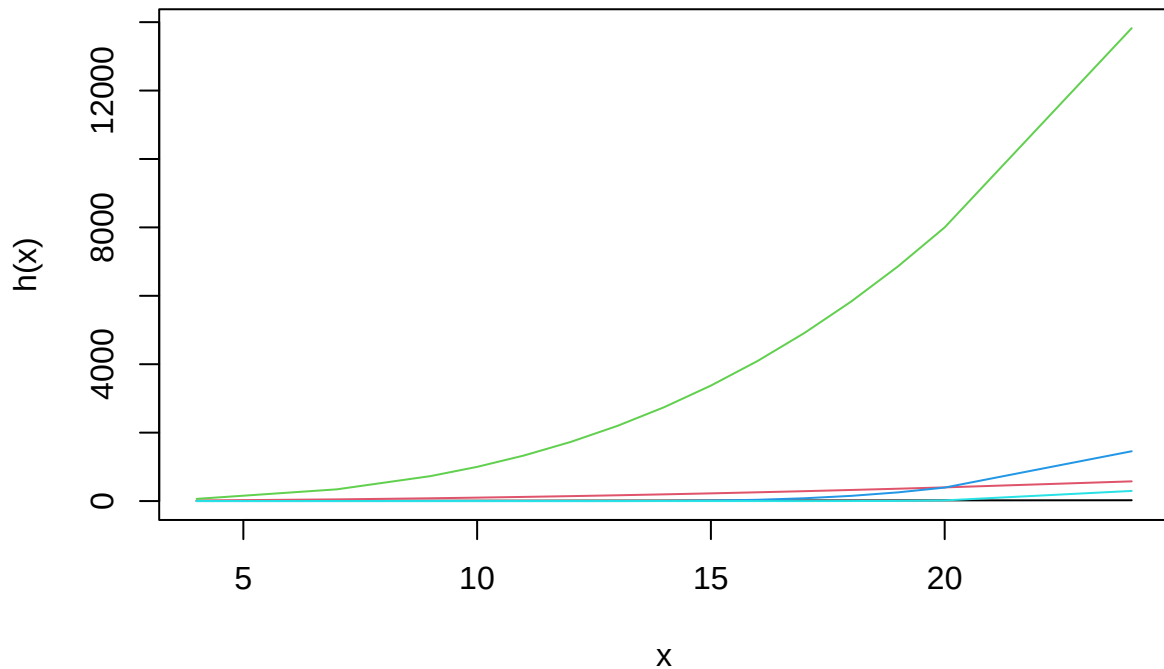
plotspl <- function(splobj,...) {
  matplot(splobj$x,splobj$X,type="l",lty=1,
          xlab="x",ylab="h(x)",...)
  abline(v=splobj$xquantiles,lty=3,col=gray(0.5))
}

# create basis functions
spl <- make_basis(train$speed)

#spl <- lecturespl(train$speed)
spl$x <- train$speed

# visualize basis functions
plotspl(spl)

```



2. Use these basis functions within `lm()` to predict the training data of the response. Plot the training data (dist versus speed) and visualize the predictions.

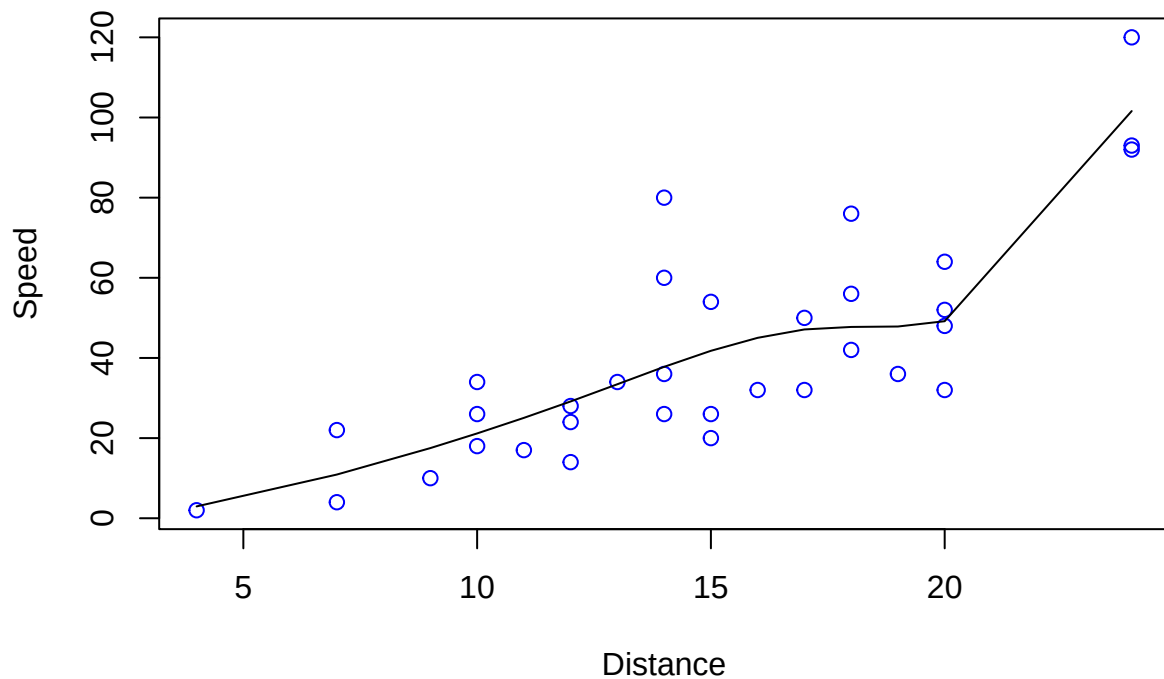
```

df_train <- data.frame(y = train$dist, spl$x)
lm1 <- lm(y ~ ., data = df_train)

train_pred <- predict(lm1, newdata=df_train)
plot(train$speed, train$dist, col="blue", type="p",
     main="Training Data with Predictions",
     xlab="Distance",
     ylab="Speed")
lines(train$speed, train_pred)

```

Training Data with Predictions

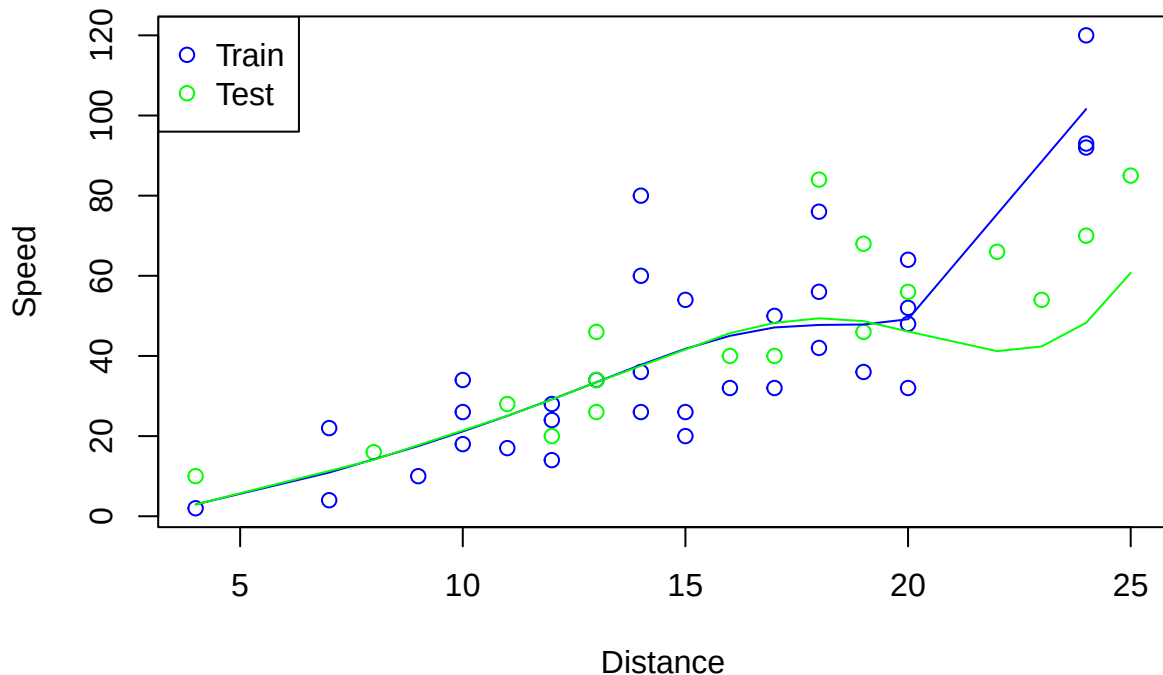


3. Use the model to predict the response from the test set observations. Visualize training and test set in one plot, using different colors, and present in this plot also the predictions for training and test set.

```
test_spl <- make_basis(test$speed)
df_test <- as.data.frame(test_spl$X)
test_pred <- predict(lm1, newdata = df_test)

plot(train$speed, train$dist, col="blue", type="p",
      main="Train and Test Sets with Predictions",
      xlab="Distance",
      ylab="Speed",
      xlim=c(min(cars$speed), max(cars$speed))
)
points(test$speed, test$dist, col="green")
lines(train$speed, train_pred, col="blue")
lines(test$speed, test_pred, col="green")
legend("topleft", legend=c("Train", "Test"), col=c("blue", "green"), pch=1)
```

Train and Test Sets with Predictions



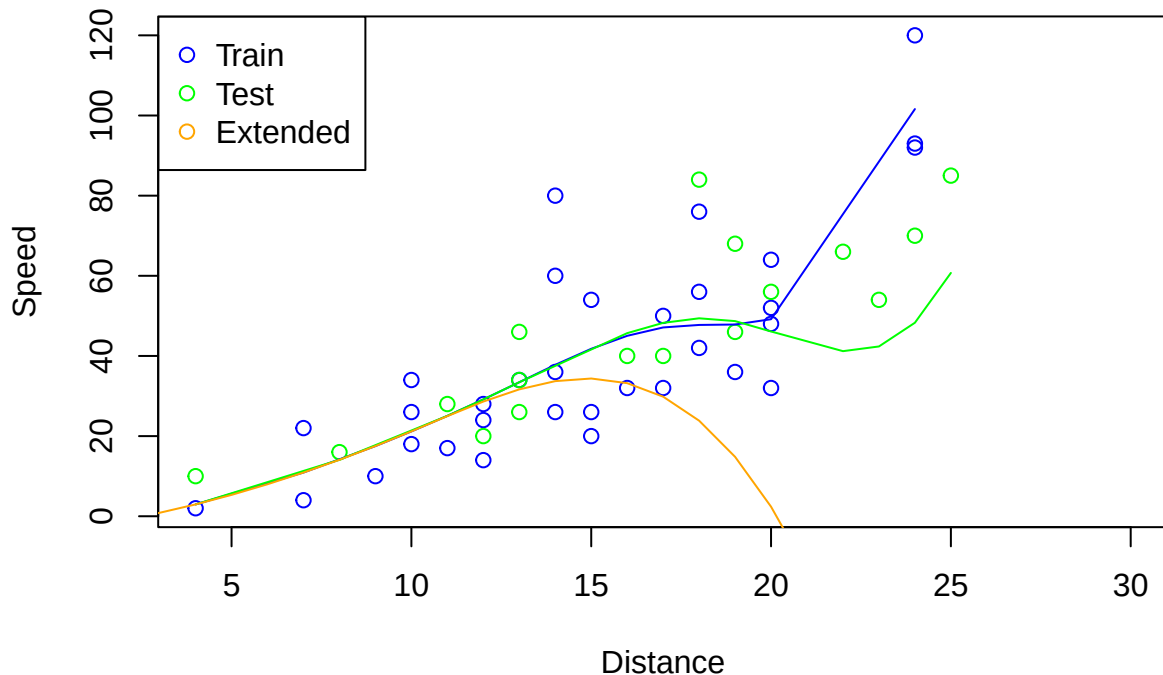
The predictions were about the same until past the 2nd knot (17.333), where they then significantly diverge between the train and test sets.

4. Generate new speed data with `seq(0,30)`. Thus, we extend the range of the explanatory variable. Use the model derived above to obtain predictions, and present those in the plot from 3. (by first extending the x-range).

```
new_X <- seq(0,30)
new_spl <- make_basis(new_X)
df_new <- as.data.frame(new_spl$X)
new_pred <- predict(lm1, newdata = df_new)

plot(train$speed, train$dist, col="blue", type="p",
     main="Train and Test Sets with Predictions and Extended X",
     xlab="Distance",
     ylab="Speed",
     xlim=c(min(cars$speed), 30)
)
points(test$speed, test$dist, col="green")
lines(train$speed, train_pred, col="blue")
lines(test$speed, test_pred, col="green")
lines(new_X, new_pred, col="orange")
legend("topleft", legend=c("Train", "Test", "Extended"),
     col=c("blue", "green", "orange"),
     pch=1)
```

Train and Test Sets with Predictions and Extended X



The predictions on the extended dataset is in line with the others until the 1st knot (12.667), where it then gets driven down.

5. You might realize that the predictions from 4. are non-sense. The problem is that `lecturespl()` constructs the knots on the quantiles of the input variable. However, we should use the knots that have been constructed for the training data. Thus, modify the function accordingly, compute the predictions again, and visualize the new results.

```
lecturespl_mod1 <- function(x, nknots=2, M=4, splobj=NULL) {
  # nknots ... number of knots -> placed at regular quantiles
  # M ... M-1 is the degree of the polynomial
  n <- length(x)
  # X will not get an intercept column
  X <- matrix(NA, nrow=n, ncol=(M-1)+nknots)
  for (i in 1:(M-1)){
    X[,i] <- x^i
  }

  # now the basis functions for the constraints:
  if (is.null(splobj)) {
    quant <- seq(0,1,1/(nknots+1))[c(2:(nknots+1))]
    qu <- quantile(x,quant)
  } else { # new addition
    quant <- splobj$quantiles
    qu <- splobj$xquantiles
  }

  for (i in M:(M+nknots-1)) {
    X[,i] <- ifelse(x-qu[i-M+1]<0,0,(x-qu[i-M+1])^(M-1))
  }
  list(X=X,quantiles=quant,xquantiles=qu)
}
```

```

}

make_basis_mod <- function(x, nknots=2, M=4, splobj=NULL) {
  out <- lecturespl_mod1(x, nknots, M, splobj) # adjusted arguments
  X <- out$X
  colnames(X) <- paste0("b", seq_len(ncol(X)))
  list(X = X, info = out)
}

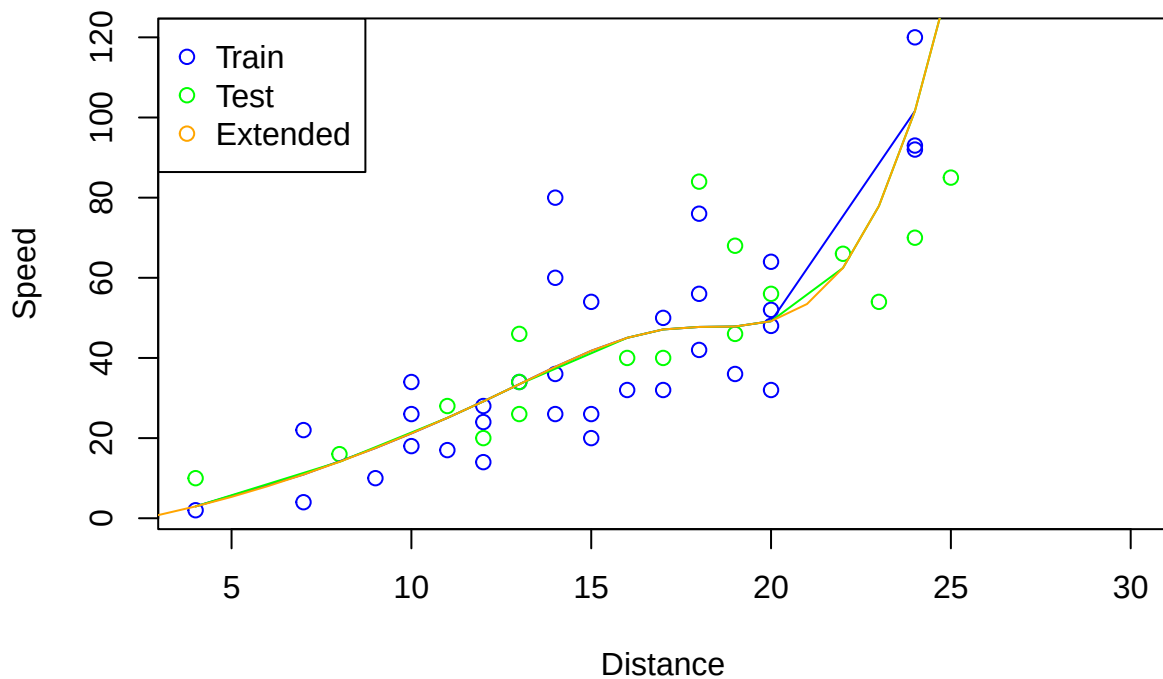
# test
test_spl <- make_basis_mod(x=test$speed, nknots=2, M=4, splobj=spl$info)
df_test <- as.data.frame(test_spl$X)
test_pred <- predict(lm1, newdata = df_test)

# extended
new_spl <- make_basis_mod(x=new_X, nknots=2, M=4, splobj=spl$info)
df_new <- as.data.frame(new_spl$X)
new_pred <- predict(lm1, newdata = df_new)

# plotting
plot(train$speed, train$dist, col="blue", type="p",
      main="Train and Test Sets with Predictions and Extended X",
      xlab="Distance",
      ylab="Speed",
      xlim=c(min(cars$speed), 30),
      )
points(test$speed, test$dist, col="green")
lines(train$speed, train_pred, col="blue")
lines(test$speed, test_pred, col="green")
lines(new_X, new_pred, col="orange")
legend("topleft", legend=c("Train", "Test", "Extended"),
      col=c("blue", "green", "orange"),
      pch=1)

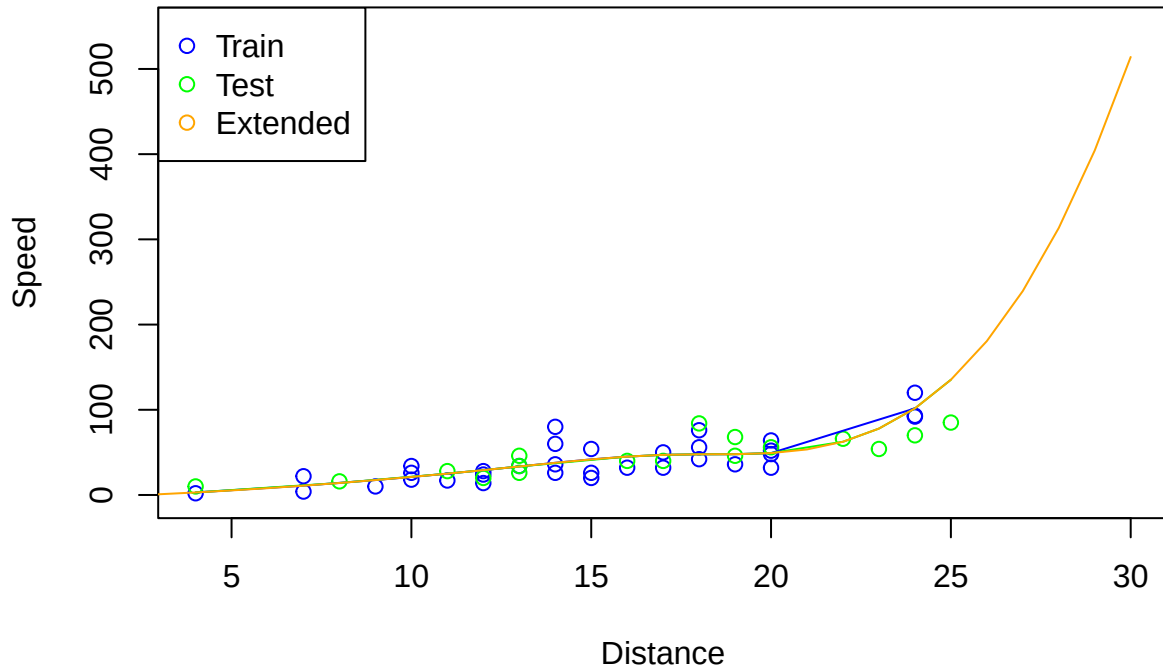
```

Train and Test Sets with Predictions and Extended X



```
# zoomed out
plot(train$speed, train$dist, col="blue", type="p",
     main="Train and Test Sets with Predictions and Extended X",
     xlab="Distance",
     ylab="Speed",
     xlim=c(min(cars$speed), 30),
     ylim=c(-5, 550)
)
points(test$speed, test$dist, col="green")
lines(train$speed, train_pred, col="blue")
lines(test$speed, test_pred, col="green")
lines(new_X, new_pred, col="orange")
legend("topleft", legend=c("Train", "Test", "Extended"),
     col=c("blue", "green", "orange"),
     pch=1)
```

Train and Test Sets with Predictions and Extended X



6. Another problem: Your predictions for low speed might look strange, as we expect for a speed of zero also a stopping distance of zero. Try to come up with a (simple) solution and present all results in one plot (training and test data, and the predictions for training, test and new x data).

```
# prevent negative response by using log transform
lm1 <- lm(log(y) ~ ., data = df_train)

# add column for intercept since missing in df_test
X_test <- cbind("(Intercept)" = 1, df_new)

# transpose multiplication of X_test with coef()
log_pred <- as.numeric(as.matrix(X_test) %*% coef(lm1))
new_pred <- exp(log_pred)

# output new_pred
new_pred # all values > 0

## [1] 5.593300e-02 1.702047e-01 4.454289e-01 1.015160e+00 2.040261e+00
## [6] 3.661656e+00 5.942315e+00 8.830104e+00 1.216616e+01 1.573856e+01
## [11] 1.935732e+01 2.292136e+01 2.646037e+01 3.015131e+01 3.409490e+01
## [16] 3.804475e+01 4.161610e+01 4.433312e+01 4.576794e+01 4.645928e+01
## [21] 4.764560e+01 5.072994e+01 5.763005e+01 7.178418e+01 1.007524e+02
## [26] 1.637501e+02 3.167076e+02 7.490975e+02 2.226759e+03 8.548996e+03
## [31] 4.356281e+04

# plotting
plot(train$speed, train$dist, col="blue", type="p",
     main="Train and Test Sets with Predictions and Extended X",
     xlab="Distance",
     ylab="Speed",
     xlim=c(min(cars$speed), 30),
```

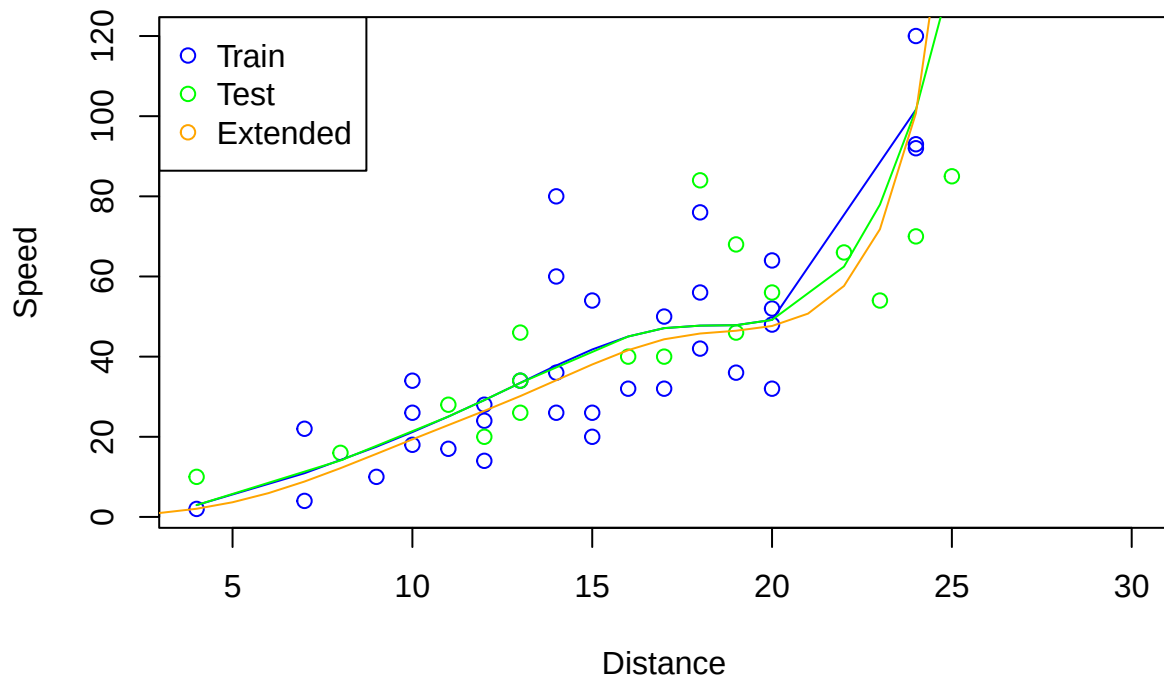


```

)
points(test$speed, test$dist, col="green")
lines(train$speed, train_pred, col="blue")
lines(test$speed, test_pred, col="green")
lines(new_X, new_pred, col="orange")
legend("topleft", legend=c("Train", "Test", "Extended"),
      col=c("blue", "green", "orange"),
      pch=1)

```

Train and Test Sets with Predictions and Extended X

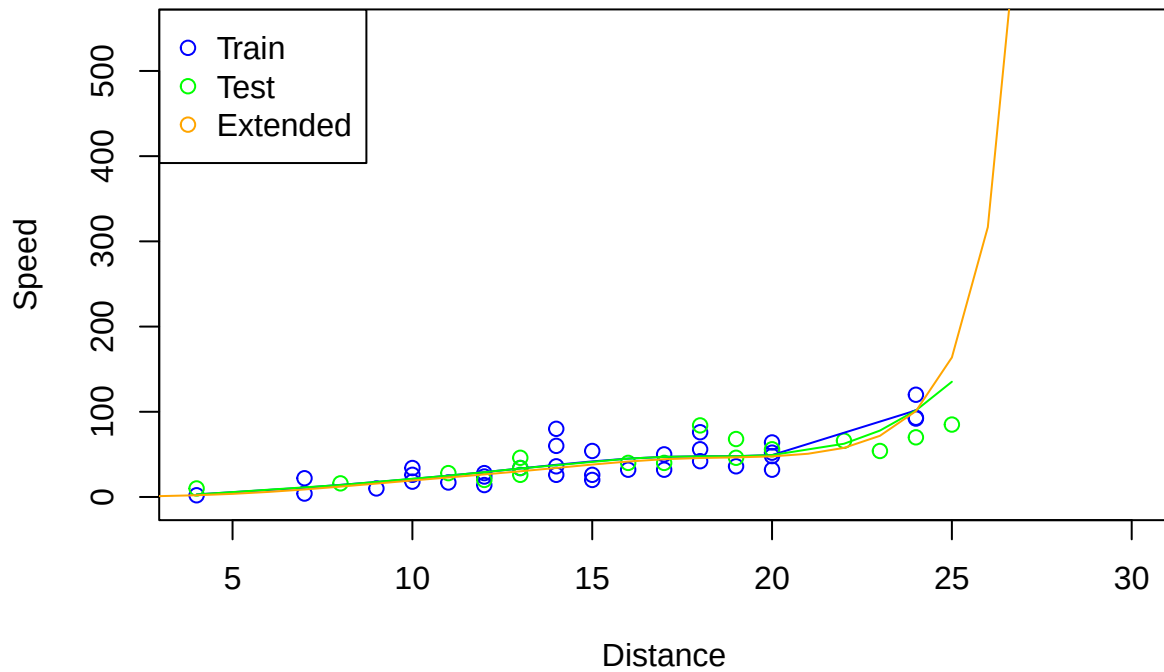


```

# zoomed out
plot(train$speed, train$dist, col="blue", type="p",
     main="Train and Test Sets with Predictions and Extended X",
     xlab="Distance",
     ylab="Speed",
     xlim=c(min(cars$speed), 30),
     ylim=c(-5, 550))
points(test$speed, test$dist, col="green")
lines(train$speed, train_pred, col="blue")
lines(test$speed, test_pred, col="green")
lines(new_X, new_pred, col="orange")
legend("topleft", legend=c("Train", "Test", "Extended"),
      col=c("blue", "green", "orange"),
      pch=1)

```

Train and Test Sets with Predictions and Extended X



The log transform fixes the issue of negative values for the response, though now the model underpredicts when $x < 25$ for the new x values compared to the train and test sets. When $x > 25$, the log-transformed model predicts much higher values compared to in task 5.